

Jupiter – Nav2 Navigation Configuration: Status & Open Problem

Date: 2026-06-12 **Author:** Logan Naidoo (with Claude as config assistant) **Purpose:** Describe the current Nav2 stack on the Jupiter robot, what works, what does not, and the specific Nav2 questions where expert input is needed. Written for review by a ROS 2 / Nav2 expert (human or model). The assistant’s Nav2 knowledge is from documentation/source, **not** hands-on tuning – treat its hypotheses as starting points to confirm, not conclusions.

1. Robot & Hardware

Item	Value
Robot	“Jupiter”, 4-wheel mecanum base
Drive mode for nav	Diff-drive style: vx + wz only, lateral strafe (vy) = 0 ; in-place pivot (wz) retained and valued. Strafe kept only for future docking.
Compute	Jetson AGX Thor, JetPack 7.1, Ubuntu 24.04, Blackwell GPU
ROS 2	Jazzy
Footprint	rectangle [[−0.175, −0.20], [0.175, −0.20], [0.175, 0.20], [−0.175, 0.20]] -> 0.35 m (X) x 0.40 m (Y) . Inscribed radius ~ 0.175 m, circumscribed ~ 0.266 m.
2D LiDAR	RPLIDAR S2E (Ethernet/UDP), mounted at 0.515 m height -> blind below ~0.5 m . ~10 Hz, /scan, frame base_laser (yaw pi, faces backward).
Depth camera	Orbbec Gemini 336 , mounted base_footprint->camera_link = (0.100, 0.000, 0.475) m, with a known ~1.3deg nose-down pitch (currently NOT modeled in the TF – deferred). Depth + pointcloud, ~15 fps.
IMU	BNO055 via ESP32/micro-ROS -> /imu/data -> corrected -> /imu/data/corrected
Wheel odom	ESP32/micro-ROS -> /odom/unfiltered

2. Localization & TF – VALIDATED HEALTHY

TF chain: map ->(AMCL)-> odom ->(EKF)-> base_footprint ->(static)-> {base_laser, camera_link, imu_link}

- **EKF** (robot_localization, ekf_odom.yaml): fuses wheel vx, vy, vyaw + IMU vyaw -> odom->base_footprint. two_d_mode: true, 50 Hz.
- **AMCL** (nav2_amcl): scan-matches S2E /scan against the static map -> map->odom. OmniMotionModel, likelihood_field, max_beams 120, particles 500-3000, update_min_d 0.05, update_min_a 0.1, transform_tolerance 1.0.
- **Static map:** maps/apartment_s2e_v2.yaml via nav2_map_server.

Empirical validation (2026-06-12): Teleop pure-forward 2.0 m with Nav2 OFF. odom->base_footprint reported **+0.093 m lateral / +2.9deg yaw** over 2 m, which *agrees* with the slight observed physical drift. Conclusion: **wheel+IMU odometry and the EKF are healthy and are NOT the problem.** /scan measured steady at **9.99 Hz**. AMCL map->odom stable. **This rules out localization/odometry as a cause of the navigation failure below.**

3. Costmaps

Both costmaps: resolution 0.05 m, transform_tolerance 0.5, footprint as above.

Global costmap (40x40 m, origin -20,-20, track_unknown_space: true)

Plugins: static_layer -> obstacle_layer -> nvblox_layer -> inflation_layer - static_layer: the pre-built apartment occupancy grid. - obstacle_layer: S2E /scan, raytrace 8.0 m, obstacle max 7.5 m, decay_time 10.0. - nvblox_layer: Orbbec->ESDF slice, max_obstacle_distance 1.0, inflation_distance 0.0. - inflation_layer: inflation_radius 0.55, cost_scaling_factor 5.0.

Local costmap (4x4 m rolling)

Plugins: obstacle_layer -> nvblox_layer -> inflation_layer - obstacle_layer: S2E /scan, decay_time 5.0. - nvblox_layer: same as global. - inflation_layer: inflation_radius 0.55, cost_scaling_factor 5.0.

nvblox (nvblox.launch.py, Isaac ROS nvblox)

Orbbec depth -> GPU TSDF on Thor -> 2D ESDF slice -> NvbloxCostmapLayer. Its job is to catch **low furniture (chairs, table legs, feet)** below the 0.515 m LiDAR plane. - mapping_type: static_tsdf, voxel_size 0.05, use_depth: true, use_color: true (color is unused by nav – wasted compute), use_lidar: false. - esdf_mode: '2d', esdf_slice_height: 0.30, esdf_2d_min_height: 0.10, esdf_2d_max_height: 1.80 (intent: exclude the floor, keep 0.10-1.80 m). - map_clearing_radius_m: 2.5, maximum_sensor_message_queue_length: 1. - Slice topic consumed by Nav2: /nvblox_node/static_map_slice.

[!] **SUSPECTED CONFIG BUG – UNCONFIRMED** (see §6). At runtime nvblox's own parameter dump shows esdf_slice_min_height = 0.0, esdf_slice_max_height = 1.0, esdf_slice_height = 1.0 – which do **not** match the launch values above. This suggests the launch parameter *names* (esdf_2d_min_height / esdf_2d_max_height / esdf_slice_height) may be wrong/ineffective for this installed version, so nvblox fell back to defaults that **slice from the floor (0.0 m) upward** – i.e. it may be marking the **floor itself** as an obstacle. **Needs verification against the installed nvblox source.**

4. Planner & Controller

Global planner – SmacPlanner2D (switched from NavFn 2026-06-12)

tolerance 0.25, allow_unknown true, motion_model_for_search MOORE (8-connected), cost_travel_multiplier 4.0 (push paths to passage/doorway centres), max_planning_time 2.0, simple_smoother on (w_smooth 0.3, w_data 0.2).

Local controller – RegulatedPurePursuitController (switched from MPPI 2026-06-12)

controller_frequency 10.0, desired_linear_vel 0.30. - Lookahead (velocity-scaled): lookahead_time 2.0 -> effective carrot ~ 0.30x2.0 = 0.60 m, clamped to [min 0.40, max 0.90]. - use_rotate_to_heading true, rotate_to_heading_min_angle 1.0 (~57deg), rotate_to_heading_angular_vel 1.0, max_angular_accel 3.2, allow_reversing false. - Regulated velocity: use_regulated_linear_velocity_scaling true (min_radius 0.90, min_speed 0.15), use_cost_regulated_linear_velocity_scaling true, inflation_cost_scaling_factor 5.0 (matches costmap), cost_scaling_dist 0.6. - **Collision check:** use_collision_detection true, max_allowed_time_to_collision_up_to_carrot 1.0.

Goal checker

xy_goal_tolerance 0.25, yaw_goal_tolerance 3.14 (final heading NOT enforced).

Velocity smoother

max_velocity [0.3, 0.0, 0.8] – lateral (y) hard-clamped to 0 (belt-and-suspenders diff-drive).

Recovery behaviors

drive_on_heading, assisted_teleop, wait only. **Spin and backup DISABLED** (spin fails on cluttered costmaps; backup drives blind in -X and previously hit furniture).

5. What WORKS vs What FAILS

[OK] Works (confirmed on hardware)

1. **Localization / TF / odometry** – see §2. Validated empirically.
2. **Global planning** – SmacPlanner2D finds a path through doorways/passages on the static map.
3. **Straight-line / clutter-free tracking** – robot tracks the planner centreline with RPP to a ~2.5 m forward goal, **smoothly**, with only minor (correct) heading corrections. This was fixed 2026-06-12 by lengthening the RPP carrot (0.45->0.60 m) and raising the rotate-to-heading threshold (45deg->57deg), which cured an earlier left/right “dance”.
4. **nvblox runs** on the Thor GPU and marks real low furniture.

[FAIL] Fails (the open problem)

Navigating out of a cluttered room (the lab) through a doorway into the passage.

- Goal: from lab (-2.23, -3.72) to (-1.53, -0.04) – exit doorway, into passage.
- The **global planner draws a complete path** to the goal (static map says route is clear).
- But the **local controller fails immediately**: RegulatedPurePursuitController detected **collision ahead!** from the first second, robot moves **< 0.5 m**, then loops: **collision ahead -> Controller patience exceeded -> Aborting -> clear local_costmap -> wait -> retry**, eventually **bt_navigator: Goal failed**.
- Control loop runs **~5.8 Hz** vs the 10 Hz target – **but CPU is only 2-15% (NOT saturated)**, so the missed rate is the loop *blocking/waiting*, not compute exhaustion.

The consistent pattern across many sessions: *clutter-free space = works; cluttered space (lab, dining room) = immediate “collision ahead” lock-up*. The variable that is active in clutter is the **nvblox 3D obstacle layer**.

6. Leading hypothesis (UNCONFIRMED – to be tested, not assumed)

nvblox is marking the floor and carpeting the local costmap around the robot, so RPP’s forward collision check finds an inscribed-lethal cell immediately and refuses to move.

Evidence: (a) the slice-height parameter mismatch in §3 (launch says exclude floor below 0.10 m; runtime dump says slice from 0.0 m); (b) the known ~1.3deg camera nose-down pitch makes the floor read as rising obstacle at range; (c) the global-clear / local-blocked split is consistent with the *local* nvblox layer marking cells the static-map-based global plan ignores.

This must be confirmed before any fix, via read-only checks + RViz: 1. Verify the **correct nvblox ESDF-slice parameter names** for the *installed* version. 2. In RViz, watch /nvblox_node/static_map_slice (and the local costmap) – is the **floor** being marked as obstacle around the robot? 3. **Decisive isolation test:** disable the **nvblox_layer** (both costmaps) and re-run the exact same lab-exit goal. If the robot then drives out (possibly clipping real low furniture), nvblox’s marking is confirmed as the blocker. (Prior sessions already saw: lidar-only -> robot moves but bumps the couch; nvblox-on -> robot boxes in. This is the core unresolved tension.)

Already ruled out (do not re-investigate): base odometry / EKF (§2, tested healthy); the RPP “dance” (fixed, §5); nvblox marks being purely “phantom” (Logan confirmed by ground truth that the static-occupancy marks include **real** furniture – though floor-marking from a bad slice height would be a *separate* issue layered on top).

7. Where expert assistance is needed (Nav2 questions)

The assistant is not confident on the following and explicitly requests review:

1. **nvblox ESDF slice / ground removal.** What are the correct parameter names for the 2D ESDF slice height band in the installed `isaac_ros_nvblox`? Is the floor being marked? What is the recommended way to exclude the floor for a forward-tilted RGBD camera – slice min-height, nvblox ground-plane/RANSAC removal, modeling the 1.3deg pitch in the TF, or temporal decay?
2. **RPP in tight clutter – right tool?** RPP is a pure path *tracker*: it has no obstacle avoidance of its own beyond *stop-on-collision*; it relies entirely on the **global path being collision-free** through the local costmap. In a furnished room with a 0.40 m-wide robot, is RPP the wrong controller, and should this be **MPPI/DWB** (which optimize trajectories *around* local obstacles) despite MPPI's earlier erratic open-space behavior? Or is RPP fine once the costmap is correct?
3. **RPP collision detection tuning.** Are `use_collision_detection: true + max_allowed_time_to_collision_up_to_c` 1.0 too conservative for tight spaces? What is the recommended configuration for a 0.40 m robot threading 0.7-0.8 m doorways and ~1.2 m passages?
4. **Inflation vs passable width.** With `inflation_radius` 0.55, `cost_scaling_factor` 5.0, and a 0.175 m inscribed radius, is there genuinely a valid (sub-inscribed-cost) corridor for the robot through a 0.7-0.8 m doorway and ~1.2 m passage, or does the cost field close the gap? How do we *measure* the actual passable width in the live costmap?
5. **Diagnosing global-clear / local-blocked.** Standard method to identify exactly **which layer and which cells** the local controller sees as blocking (footprint-vs-costmap visualization, costmap layer introspection) so we stop guessing which sensor source is at fault.
6. **Control loop at 5.8 Hz with idle CPU.** What blocks the controller loop when CPU is not saturated? (costmap update waiting on a transform? nvblox layer subscription latency? collision check cost?) How to profile it cleanly.
7. **Recovery strategy for a mecanum/diff-drive base** where spin and backup are unsafe – is clear-costmap + wait sufficient, or is a different recovery set advisable?

8. Reference files (in repo)

File	Role
<code>src/jupiter_bringup/config/nav2_params.yaml</code>	All Nav2 params (costmaps, RPP, Smac, AMCL, behaviors)
<code>src/jupiter_bringup/config/ekf_odom.yaml</code>	EKF fusion config
<code>src/jupiter_bringup/launch/navigation_s2e.launch.py</code>	Full nav bring-up (sensors, EKF, AMCL, Nav2, camera, nvblox)
<code>src/jupiter_bringup/launch/nvblox.launch.py</code>	nvblox node config (ESDF slice params in question)
<code>src/jupiter_bringup/launch/teleop_test.launch.py</code>	Base+lidar+EKF only (used to validate odometry)
<code>maps/apartment_s2e_v2.yaml</code>	Static apartment map

9. One-paragraph summary for a reviewer

Jupiter is a mecanum robot driven as diff-drive (vx+wz, pivot retained, no strafe) on Jetson Thor / ROS 2 Jazzy. Localization (EKF wheel+IMU odometry -> odom, AMCL S2E-LiDAR vs static map -> map) is **empirically validated healthy**. The Nav2 stack is **SmacPlanner2D** (global) + **Regulated Pure Pursuit** (local), with a hybrid costmap: S2E LiDAR `ObstacleLayer` + an **Isaac ROS nvblox** ESDF layer (Orbbec depth, for low furniture below the LiDAR plane) + inflation (0.55 m, `cost_scaling` 5.0), nvblox in **both** costmaps. **Straight-line tracking in clear space works. Navigating out of a cluttered room through a doorway fails:** the global planner finds a path, but RPP reports collision ahead immediately and never progresses. Leading (unconfirmed)

suspicion is that nvblox’s ESDF slice is mis-parameterized and marking the **floor**, boxing the robot in – but this needs confirmation, and there is an open architectural question of whether RPP (a pure tracker) is the right local controller for tight, furnished spaces versus a trajectory optimizer (MPPI/DWB).